

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Atchaya Chandran R(192521394) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Atchaya

1. Password Hashing with Salting and Key Stretching

PDF

Python

```
import hashlib
```

```
import os
```

```
def hash_password(password: str, iterations: int = 100000) -> tuple[bytes, bytes]:
```

```
    salt = os.urandom(16)
```

```
    key = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt, iterations)
```

```
    return salt, key
```

```
def verify_password(stored_salt: bytes, stored_key: bytes, password_attempt: str, iterations: int = 100000) -> bool:
```

```
    new_key = hashlib.pbkdf2_hmac('sha256', password_attempt.encode('utf-8'), stored_salt, iterations)
```

```
    return new_key == stored_key
```

Execution Example

```
salt, hash_val = hash_password("SecurePassword123")
```

```
print("Verification Result:", verify_password(salt, hash_val, "SecurePassword123"))
```

- **Salting Impact:** A salt is a cryptographically random, unique byte sequence added to each password before hashing. It ensures identical passwords produce distinct hash values, completely neutralizing precomputed lookup attacks (Rainbow Tables) and preventing bulk cracking across users.
- **Key Stretching:** Applying iterative hashing functions (e.g., PBKDF2 with iterations) artificially increases the computational cost per guess. This significantly slows down offline GPU-accelerated brute-force and dictionary attacks.
- **Performance Overhead:** While adding milliseconds of delay per login attempt is imperceptible to a legitimate user, it scales non-linearly for an attacker, rendering billions of guesses per second computationally infeasible.

2. Implementation of HMAC for Message Integrity

PDF

Python

```
import hmac
```

```
import hashlib
```

```
def generate_hmac(secret_key: bytes, message: bytes) -> str:
```

```
    return hmac.new(secret_key, message, hashlib.sha256).hexdigest()
```

```
def verify_hmac(secret_key: bytes, message: bytes, received_mac: str) -> bool:
```

```
    expected_mac = generate_hmac(secret_key, message)
```

```
    return hmac.compare_digest(expected_mac, received_mac)
```

```
# Execution Example
```

```
key = b'super_secret_shared_key'
```

```
msg = b'Transfer $1000 to Account B'
```

```
mac = generate_hmac(key, msg)
```

```
print("HMAC Valid:", verify_hmac(key, msg, mac))
```

```
print("Tampered Valid:", verify_hmac(key, b'Transfer $9000 to Account B', mac))
```

- **Simple Hashing vs. HMAC:** Simple hashing () ensures integrity only against accidental corruption; anyone can alter the message and recompute . HMAC incorporates a shared secret key () via , which ensures both data integrity and message origin authenticity. Simple key concatenations like are vulnerable to length-extension attacks, which HMAC inherently prevents.
- **Security Assessment:** HMAC guarantees that any unauthorized payload modification invalidates the tag and verifies that the message originated from a party holding the shared key.

3. Simulation of Certificate Chain Validation

PDF

Python

```
import hashlib
```

```
class Certificate:
```

```
    def __init__(self, subject: str, issuer: str, pub_key: str, is_revoked: bool = False):
```

```
        self.subject = subject
```

```
        self.issuer = issuer
```

```
        self.pub_key = pub_key
```

```
        self.is_revoked = is_revoked
```

```
        self.signature = ""
```

```
    def sign(self, issuer_private_key: str):
```

```
        data = f"{self.subject}:{self.issuer}:{self.pub_key}"
```

```
        self.signature = hashlib.sha256((data + issuer_private_key).hexdigest().encode()).hexdigest()
```

```
def validate_chain(chain: list[Certificate], crl: set[str]) -> bool:
```

```
    for i in range(len(chain)):
```

```
        cert = chain[i]
```

```
        # Check revocation (CRL check)
```

```
        if cert.subject in crl or cert.is_revoked:
```

```
            print(f"Validation failed: Certificate {cert.subject} is revoked.")
```

```
            return False
```

```
        # Determine signing entity
```

```
        if i == len(chain) - 1: # Root CA self-signed check
```

```
            issuer_cert = cert
```

```
        else:
```

```
            issuer_cert = chain[i + 1]
```

```
        if cert.issuer != issuer_cert.subject:
```

```
            print(f"Chain broken between {cert.subject} and {cert.issuer}")
```

```
            return False
```

```
    print("Certificate chain successfully validated to trusted Root CA.")
```

```
    return True
```

- **Trust Chain Mechanics:** Validation builds a path from the end-entity certificate through intermediate CAs up to a trusted Root CA. The public key extracted from the parent certificate verifies the signature on the child certificate.
- **Detection of Invalid/Revoked Certs:** Certificates are rejected if signatures do not verify, expiration dates are exceeded, or identity strings are listed on Certificate Revocation Lists (CRLs) or checked via Online Certificate Status Protocol (OCSP).

4. Replay Attack Simulation and Prevention

PDF

Python

```
import time
```

```
import secrets
```

```
import hashlib
```

```
class Server:
```

```
    def __init__(self, key: bytes):
```

```
        self.key = key
```

```
        self.used_nonces = set()
```

```
    def verify_request(self, message: str, timestamp: float, nonce: str, mac: str) -> bool:
```

```
        # Check time window (5-second threshold)
```

```
        if abs(time.time() - timestamp) > 5.0:
```

```
            return False # Expired request
```

```
        # Check nonce reuse
```

```
        if nonce in self.used_nonces:
```

```
            return False # Replay attack detected
```

```
        # Verify integrity
```

```
        payload = f"{message}:{timestamp}:{nonce}".encode()
```

```
        expected_mac = hashlib.pbkdf2_hmac('sha256', payload, self.key, 1000).hex()
```

```
        if expected_mac == mac:
```

```
            self.used_nonces.add(nonce)
```

```
            return True
```

```
        return False
```

```
# Simulation
```

```
secret = b'server_client_key'
```

```
server = Server(secret)
```

```
ts = time.time()
```

```
nonce = secrets.token_hex(8)
```

```
payload_str = f"PAYMENT_REQUEST:{ts}:{nonce}".encode()
```

```
mac = hashlib.pbkdf2_hmac('sha256', payload_str, secret, 1000).hex()
```

```
print("Initial Request:", server.verify_request("PAYMENT_REQUEST", ts, nonce, mac))
```

```
print("Replayed Request:", server.verify_request("PAYMENT_REQUEST", ts, nonce, mac))
```

- **Security Improvement:** Adding unique single-use nonces and short-lived timestamps ensures that an eavesdropper capturing network packets cannot reuse them.
- **Limitations:** Timestamp mechanisms depend on strict clock synchronization across systems (e.g., NTP), making them vulnerable to clock skew. Nonce tracking requires maintaining server-side state storage, creating potential memory scale issues under high traffic.

5. Performance Comparison of Digital Signature Algorithms

PDF

Python

```
import time
```

```
from cryptography.hazmat.primitives.asymmetric import rsa, ec
```

```
from cryptography.hazmat.primitives import hashes
```

```
# RSA 2048 Benchmarking
```

```
t0 = time.time()
```

```
rsa_priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
```

```
rsa_keygen_time = time.time() - t0
```

```
data = b"Authentication payload"
```

```
t0 = time.time()
```

```
rsa_sig = rsa_priv.sign(data, padding=rsa.PKCS1v15(), algorithm=hashes.SHA256())
```

```
rsa_sign_time = time.time() - t0
```

```
t0 = time.time()
```

```
rsa_priv.public_key().verify(rsa_sig, data, padding=rsa.PKCS1v15(), algorithm=hashes.SHA256())
```

```
rsa_verify_time = time.time() - t0
```

```
# ECDSA (Secp256r1) Benchmarking
```

```
t0 = time.time()
```

```
ec_priv = ec.generate_private_key(ec.SECP256R1())
```

```
ec_keygen_time = time.time() - t0
```

```
t0 = time.time()

ec_sig = ec_priv.sign(data, ec.ECDSA(hashes.SHA256()))

ec_sign_time = time.time() - t0


t0 = time.time()

ec_priv.public_key().verify(ec_sig, data, ec.ECDSA(hashes.SHA256()))

ec_verify_time = time.time() - t0


print(f"RSA-2048 | KeyGen: {rsa_keygen_time:.4f}s | Sign: {rsa_sign_time:.4f}s | Verify: {rsa_verify_time:.4f}s")
print(f"ECDSA-256 | KeyGen: {ec_keygen_time:.4f}s | Sign: {ec_sign_time:.4f}s | Verify: {ec_verify_time:.4f}s")
```

Metric / Algorithm	RSA (2048-bit)	ECDSA (256-bit / SECP256R1)
Key Generation Time	Slow (requires large prime search)	Extremely Fast
Signing Speed	Slower	Faster
Verification Speed	Fast (small public exponent)	Moderately Slower than RSA
Key / Signature Size	Large (2048 bits / 256 bytes)	Small (256 bits / ~64 bytes)
Best Application	Static code signing, certificate verification	Mobile devices, TLS handshakes, IoT, Blockchain